LUMEN

Beginner-Friendly Code Walkthrough

Below is a beginner-friendly walkthrough of the Python code in LUMEN. The main idea is:

1. Make or load example student and course data.
2. Turn each student/course combination into numbers.
3. Train a model to estimate a “fit score.”
4. Use those scores to rank courses.
5. Show reasons for the scores in the web app.

A function is a named set of instructions. When another part of the program calls that function, Python runs those instructions.

app.py — the web app

This file creates the screen you interact with and connects the other parts of the project.

Imports and setup

- from __future__ import annotations enables a newer way to write type hints—notes describing the kind of value a function expects or returns.
- ast is used to safely turn text that looks like a Python list back into a list.
- Path helps build file and folder paths.
- pandas is used for table-like data.
- streamlit is the library that creates the web interface.
- The imports beginning with lumen bring in the project’s data, model, and recommendation functions.

ROOT, DATA_DIR, and ARTIFACTS_DIR store the locations of the project, its data folder, and its saved-model folder. Building paths this way helps the app find files no matter which folder it was launched from.

ensure_pipeline()

The @st.cache_resource line tells Streamlit to keep the loaded model and data around between screen updates. That avoids reloading them every time you change a control.

The function:

1. Loads the student, course, and training-pair tables.
2. Checks whether the saved model file exists.
3. Trains a model if the file is missing.
4. Loads the model.

5. Returns the data and model so the rest of the app can use them.

main()

This function builds the page.

- st.set_page_config(...) sets the browser page title, icon, and layout.
- st.title(...) and st.caption(...) display the heading and description.
- st.columns([1, 2]) creates two columns: a narrower one for the student profile, and a wider one for the recommendations.

In the left column, the app offers two ways to create a profile:

- Pick from dataset: choose an existing sample student.
- Custom profile: choose a major, year, GPA, and completed courses.

The custom profile is assembled as a pandas Series, which behaves like a small table row. The app then displays the current profile.

Completed courses are stored as text in the CSV files. ast.literal_eval tries to convert text such as "['CS101']" into a real list. If the text is malformed, the except block catches the parsing error so the app can continue.

The recommendation slider chooses how many results to show. Clicking Refresh synthetic cohort generates new sample students and training pairs, saves them as CSV files, retrains the model, clears Streamlit's cached data, and reruns the page.

In the right column, recommend_courses(...) generates ranked results. If no courses remain, the app displays a message and stops drawing the rest of the page. Otherwise, it draws a bordered card for each recommendation with the course details, score, and explanation.

At the bottom, the app displays a brief note about the software stack and says the data is synthetic. The final if __name__ == "__main__": block means "run main() when this file is launched directly."

lumen/data/synthetic.py — sample courses, students, and scores

This file makes the demo data.

Lists and mappings

MAJORS contains the five sample majors. DEPARTMENTS maps each major to a short department code, such as Computer Science to CS.

COURSE_CATALOG is a list of courses. Each course is described by:

- Course ID and name
- Department
- Difficulty level
- Credits
- Prerequisite course IDs

For example, the catalog says Data Structures is a CS course, has difficulty 2, is worth 4 credits, and requires Intro to Programming.

courses_dataframe()

This function turns the course catalog into a pandas table, with one row per course. A DataFrame is pandas' name for a table with rows and columns.

_major_dept(major)

Looks up the department code for a major. The underscore at the start of the name is a convention meaning "intended for internal use." It doesn't prevent other code from calling it.

_prereqs_met(completed, prereqs)

Checks whether every prerequisite is in the student's completed-course set. It returns True if all are present and False if at least one is missing.

A set is like a list but is designed for fast membership checks such as "is CS101 in the completed courses?"

_synthetic_fit(...)

This function creates the example "ground truth" fit score. "Ground truth" here means the score used as the answer during training; it is generated by rules, not measured from real student results.

It starts at 0.35, then adjusts the score:

- Adds points when the course is in the student's major department.
- Gives math courses a smaller boost for students outside the math department.
- Adds points if prerequisites are met, or subtracts points if they aren't.
- Compares the difficulty with a readiness value based on year and GPA.
- Applies an extra penalty for a difficult course when GPA is below 2.7.
- Adds a small random variation.
- Uses np.clip to keep the final score between 0 and 1.

rng is a random-number generator. Passing it into the function allows the caller to control randomness with a seed.

generate_students(n, seed=42)

Creates n sample students. A seed makes random choices repeatable: the same seed should generate the same sample data.

For each student, the function randomly chooses a major and year, generates a GPA, and selects some completed courses. It limits the course selection based partly on year and adds the selected courses' direct prerequisites. Finally, it returns all students as a DataFrame.

build_training_pairs(students, seed=42)

Combines every student with every course. For each combination, it calculates the synthetic fit score and writes a row containing the student details, course details, prerequisites, and score.

With 200 students and 18 courses, this creates 3,600 rows.

save_default_dataset(data_dir, n_students=200, seed=42)

Creates the data folder if needed, generates the students, builds the course catalog and training pairs, and saves all three tables as CSV files.

lumen/features.py — turning details into model inputs

Machine-learning models generally need numeric inputs. This file converts the student and course information into a consistent collection of numbers called features.

MAJOR_COLS, FEATURE_NAMES, and HUMAN_LABELS

MAJOR_COLS creates one feature name for each major. These columns work like checkboxes: the student's major gets 1, and the others get 0.

FEATURE_NAMES lists every feature in the order the model expects. Besides major, year, and GPA, the list includes department match, prerequisites, course difficulty, credits, and whether the course is already completed.

HUMAN_LABELS maps technical feature names to friendlier labels for explanations, such as gpa to "Your GPA."

_parse_completed(val)

CSV files store lists as text. This function converts those values into Python lists:

- Missing values become an empty list.
- An existing list is returned as-is.
- Text beginning with [is interpreted as a list.
- Other text is split on the | character.

_prereqs_met(completed, prereqs)

This is another prerequisite check, used here when building model features. It returns whether all prerequisites are in the completed set.

row_to_features(...)

This is the main conversion function. It receives a student and a course, then creates a dictionary of numeric values:

- One-hot major values: 1 for the student's major, 0 for the others.
- GPA and year.
- Whether the course department matches the student's major.
- Whether it's a math course.
- Whether prerequisites are met and how many there are.
- How many completed courses are in the same department.
- Course difficulty and credits.
- A difficulty-versus-readiness value.
- Whether the course was already completed.

"Difficulty versus readiness" is calculated as course difficulty minus an approximate readiness score based on year and GPA. A positive value means the course is estimated to be more challenging than the student's readiness level.

The function looks up course departments so it can count completed courses in the same department. It returns a dictionary whose feature names match FEATURE_NAMES.

dataframe_to_feature_matrix(df)

This converts a whole table of student/course examples into a numeric DataFrame for training. It loops through the rows, parses the completed-course and prerequisite lists, calls row_to_features(...) for each row, then builds a table in the required feature order.

student_course_row(student, course)

This does the same conversion for one student and one course. The recommendation code uses it when it needs to score a single candidate course.

lumen/model/gbm.py — choosing the model type

create_fit_regressor(random_state=42)

This function tries to load LightGBM and create a LGBMRegressor. A regressor predicts a number—in this case, a fit score.

The model settings control how it learns. For example:

- n_estimators=200 sets the number of boosting rounds.
- learning_rate=0.05 controls how strongly each round updates the model.
- max_depth=6 limits tree depth.
- random_state helps make training repeatable.

If LightGBM can't be imported or its native library can't load, the except block issues a warning and creates scikit-learn's HistGradientBoostingRegressor instead. Both are tree-based approaches, but they are different implementations.

lumen/model/train.py — training and saving

load_or_create_data(data_dir)

Checks whether the student CSV exists. If it doesn't, it generates the default dataset. Then it reads the student, course, and training-pair CSVs and returns them as DataFrames.

train_model(pairs, model_path, random_state=42)

This function trains the model:

1. Converts the training-pair table into numeric features with dataframe_to_feature_matrix(...).
2. Takes fit_score as the target answer the model should learn.
3. Splits the rows into training and test portions. The model learns from the training portion; the test portion is held aside to estimate how well it predicts unseen rows.
4. Trains a simple Ridge regression model as a baseline and prints its prediction error.
5. Creates and trains the tree-based model.
6. Prints which backend was used, plus RMSE and R^2 measurements.
7. Creates the output folder if needed and saves the model, feature names, and backend name using joblib.

RMSE summarizes prediction error; lower is better. R^2 compares predictions with a simple "predict the average" reference; higher is generally better.

main()

Finds the project's data and artifact folders, loads or creates the data, then trains and saves the model. The final if __name__ == "__main__": block lets you run this file directly with python -m lumen.model.train.

lumen/recommend.py — scoring and ranking courses

load_model(artifacts_dir)

Loads the saved joblib file and returns the model stored inside it.

recommend_courses(...)

This function produces recommendations for one student:

1. Parses the student's completed courses.
2. Copies the course table so it can filter candidates without changing the original.
3. Removes completed courses when exclude_completed is True.
4. For each remaining course, builds a one-row feature table and gets an explanation and score from explain_prediction(...).
5. Collects course details, score, and explanation bullets.
6. Sorts results by score, highest first.
7. Returns the first top_n results.

The score is clipped between 0 and 1 in the explanation code. The app displays it as a percentage, such as 0.73 becoming 73%.

lumen/explain.py — explanations for predictions

_EXPLAINERS and _get_explainer(model)

SHAP needs an explainer object to explain a model. _get_explainer stores one per model in a dictionary, so it can reuse the explainer rather than rebuilding it for every course.

ExplanationBullet

This is a small data structure for one explanation item. It holds:

- label: the human-readable feature name.
- direction: whether the feature raises, lowers, or has a neutral effect on the score.
- magnitude: the size of the estimated influence.
- detail: the sentence shown to the user.

_direction(shap_val)

Converts a SHAP value into a simple direction:

- Above 0.01 means "increases."
- Below -0.01 means "decreases."
- Between those values means "neutral."

A SHAP value is an estimate of how much a feature moved this particular prediction relative to the model's baseline.

_detail_for_feature(name, value, shap_val)

Creates an explanation sentence. Some features have custom messages—for example, whether prerequisites are met or how course difficulty compares with readiness. Other features use a general sentence saying that the feature raises or lowers the fit score.

explain_prediction(model, student, course, top_k=5)

This function explains one student/course score:

1. Converts the student and course into a one-row feature table.
2. Gets the model's prediction and clips it to the 0–1 range.
3. Gets SHAP values for that prediction.
4. Sorts the features by the absolute size of their SHAP values, largest first.
5. Converts up to top_k meaningful influences into explanation bullets.
6. If no feature has a large enough influence, returns a generic "Overall profile" explanation.

It returns both the score and the explanation bullets.

format_summary(course_name, score, bullets)

Turns the score and bullets into Markdown text for Streamlit. It formats the score as a percentage and adds an up arrow for features that increase the score, a down arrow for features that decrease it, or a dot for neutral features.

lumen/__init__.py and the other __init__.py files

These files are empty or contain only the project version. Their main purpose is to mark folders as Python packages, so Python can import modules from them. The top-level one sets the version to 0.1.0.

Two important details about what the code means

- The generated fit score is a demo score based on hand-written rules. The model learns to imitate those rules; it has not learned from real student outcomes.
- The recommender removes courses the student already completed, but it doesn't strictly remove courses for which prerequisites are missing. Those courses can still be recommended with a lower score.